The `scalar_mul_secret` bit-length timing leak, before and after

Study `research/scalar_mul_ct_timing_20261022` · 2026-10-22 · audit finding 7

The leak.

- `ecc::ct::scalar_mul_secret` sends every curve other than secp256k1 and P-256 to `Point::scalar_mul_ct`, the affine `BigUint` Montgomery ladder. SM2, GOST, EC-KCDSA and EC-ElGamal also call that ladder directly.
- The ladder started at `r0 = 0`. Through every leading zero bit of the secret scalar it took the `0 + P` and `2·0` shortcuts, so wall time revealed the scalar's bit length. Nonce bit lengths are what lattice attacks on signatures use.

The fix.

- The ladder runs on `k + 2^L` over `L + 1` bits, starting from `(P, 2P)`, and subtracts `[2^L]P`. The result is exactly `[k mod 2^L]P` for every point, including points off the prime-order subgroup.
- `[2^L]P` depends only on the public `P` and `L`. In the shipped version (run 2, amendment 1) it comes from the Jacobian `Point::scalar_mul`. In the first version (run 1) it was `L` affine doublings.
- The unit test `ladder_work_does_not_depend_on_scalar_length` counts ladder steps that hold the identity. It fails on the old ladder and finds none on the new one.

Arms and procedure.

- Before: `main` plus the driver, `f7f455e7c`. A′: the same binary, the A/A control.
- After: `360f28a3d` in run 2, `1dce1c654` in run 1.
- Each run: rotated rounds, 3 repetitions per process, one process per arm per round, single-threaded under `tools/isolated_bench.py`.
- Point: the generator. Scalars: 8 classes of leading zero bits, 8 scalars each.
- Build: cargo build --release (default profile) rustc 1.97.0 (2d8144b78 2026-07-07). Host: Intel Xeon @ 2.10 GHz, 4-vCPU KVM guest.

1. Result (run 2: the shipped fix)

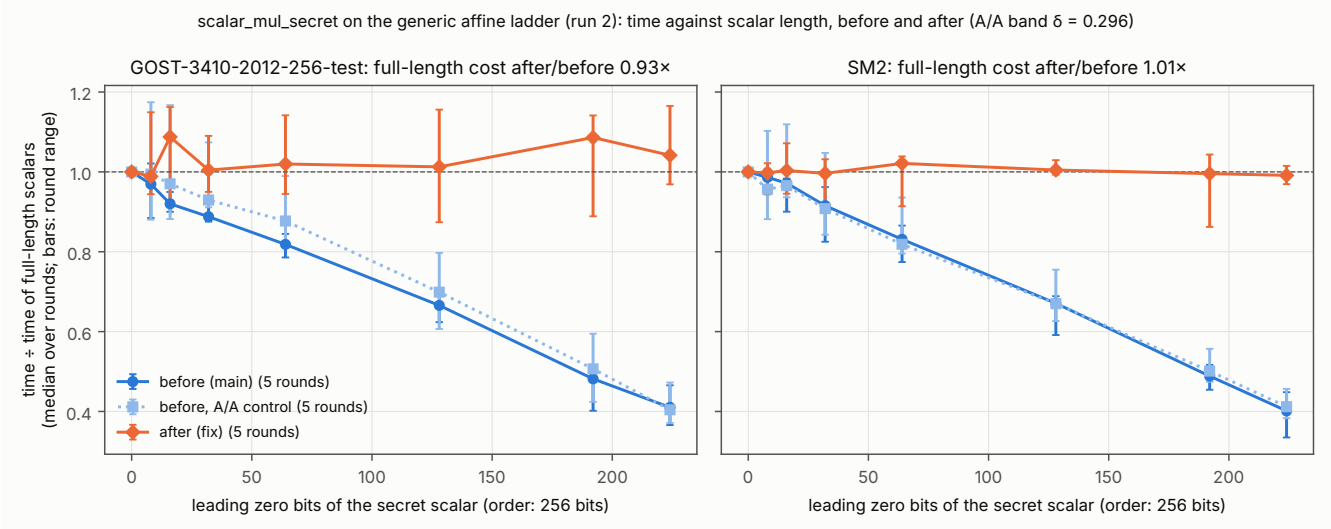


Figure 1: Run 2. Time of `scalar_mul_secret` per leading-zero class, as a fraction of the full-length class: median over rounds, with bars for the range over rounds. Dashed line: no dependence on length.

curve	R = t(224 zeros)/t(full), before, worst round	flatness F after, worst round	A/A band δ	full-length ms, before ⟶ after (round 1)	cost after/before, median	range over rounds
GOST-3410-2012-256-test	0.465	0.165	0.296	87.1 ⟶ 75.01	0.929	0.861–1.089
SM2	0.448	0.138	0.296	99.53 ⟶ 100.45	1.009	0.964–1.038

Verdicts. These are the frozen rules of `PROTOCOL.md`.

- Leak present before (`R < 0.9` in every round, on both curves): yes.
- Coarse leak closed after (`F ≤ δ` in every round, on both curves): yes.
- Pinned result digests identical across all runs: yes.
- Rounds: 5. Contended runs left out: none.

2. Run 1: `[2^L]P` by affine doublings

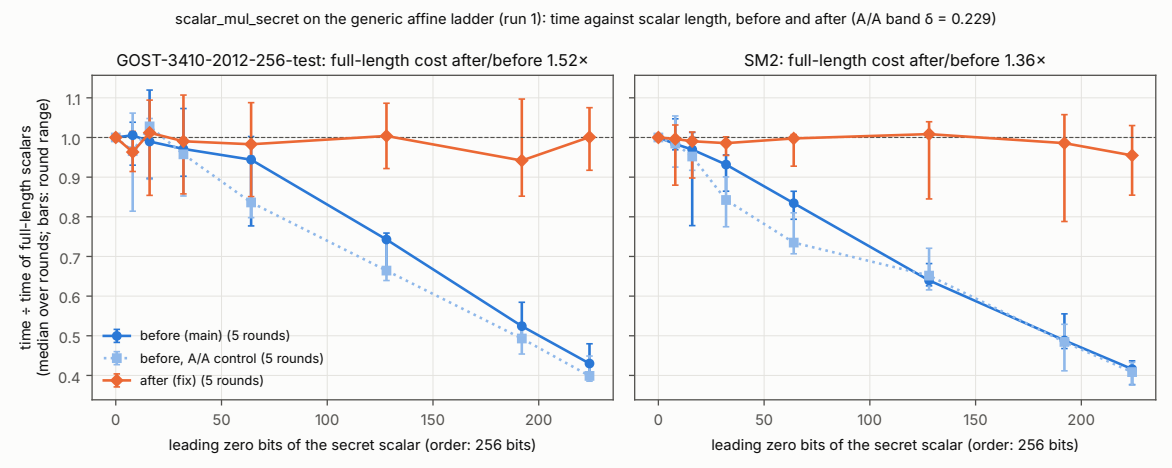


Figure 2: Run 1, the first version of the fix, before amendment 1.

curve	R = t(224 zeros)/t(full), before, worst round	flatness F after, worst round	A/A band δ	full-length ms, before ⟶ after (round 1)	cost after/ before, median	range over rounds
GOST-3410-2012-256-test	0.48	0.149	0.229	70.21 ⟶ 108.78	1.517	1.294–1.549
SM2	0.437	0.212	0.229	98.02 ⟶ 130.36	1.359	1.29–1.422

Verdicts.

- Leak present before (`R < 0.9` in every round, on both curves): yes.
- Coarse leak closed after (`F ≤ δ` in every round, on both curves): yes.
- Pinned result digests identical across all runs: yes.
- Rounds: 5. Contended runs left out: none.

3. Per round

Run 2.

GOST-3410-2012-256-test, round	R: before, A′, after	F: before, A′, after	A/A band	cost	full-length ms before, after
1	0.366, 0.431, 1.042	0.634, 0.569, 0.126	0.093	0.861	87.1, 75.01
2	0.465, 0.401, 1.044	0.535, 0.599, 0.088	0.296	0.929	83.09, 77.15
3	0.41, 0.473, 0.969	0.598, 0.527, 0.051	0.201	1.049	80.17, 84.11
4	0.41, 0.371, 1.165	0.59, 0.629, 0.165	0.138	0.885	80.78, 71.49
5	0.432, 0.404, 0.972	0.568, 0.596, 0.126	0.11	1.089	78.84, 85.88

SM2, round	R: before, A′, after	F: before, A′, after	A/A band	cost	full-length ms before, after
1	0.401, 0.412, 0.969	0.599, 0.588, 0.138	0.019	1.009	99.53, 100.45
2	0.335, 0.383, 0.987	0.665, 0.617, 0.072	0.132	0.964	100, 96.41
3	0.448, 0.456, 0.992	0.552, 0.544, 0.092	0.087	1.027	95.58, 98.18
4	0.433, 0.427, 1.015	0.567, 0.573, 0.032	0.101	1.038	95.25, 98.87
5	0.382, 0.403, 0.991	0.618, 0.597, 0.061	0.118	1.007	98.38, 99.03

Run 1.

GOST-3410-2012-256-test, round	R: before, A′, after	F: before, A′, after	A/A band	cost	full-length ms before, after
1	0.466, 0.449, 1.036	0.534, 0.551, 0.036	0.077	1.549	70.21, 108.78
2	0.43, 0.399, 1.075	0.57, 0.601, 0.107	0.113	1.294	76.92, 99.57
3	0.427, 0.43, 0.952	0.573, 0.57, 0.105	0.13	1.473	72.99, 107.54
4	0.402, 0.394, 1.001	0.598, 0.606, 0.149	0.08	1.517	67.4, 102.24
5	0.48, 0.385, 0.917	0.52, 0.615, 0.148	0.171	1.518	66.5, 100.96

SM2, round	R: before, A′, after	F: before, A′, after	A/A band	cost	full-length ms before, after
1	0.416, 0.408, 1.001	0.584, 0.592, 0.031	0.197	1.33	98.02, 130.36
2	0.377, 0.408, 0.855	0.623, 0.592, 0.212	0.126	1.422	95.11, 135.26
3	0.42, 0.412, 1.03	0.58, 0.588, 0.058	0.097	1.29	93.54, 120.64

4	0.402, 0.433, 0.942	0.598, 0.567, 0.079	0.099	1.359	87.75, 119.25
5	0.437, 0.376, 0.955	0.563, 0.624, 0.12	0.229	1.414	88.84, 125.59

4. Scope

- **Measured.** Coarse wall time on this host, for two curves and their generators.
- **Not addressed.** The `num-bigint` field arithmetic is not constant-time at the limb level (`SECURITY.md`). Its per-operation timing is untouched here, and so are cache, power and EM channels.
- **Small-order points.** On such points, for example attacker-chosen off-curve points, the registers can still meet the identity mid-ladder. Rejecting those points is the job of point validation.
- **Public point.** `[2^L]P` is computed in variable time from `P`, so `P` must be public. It is for every caller: generators, public keys and ciphertext points.
- **Cost.** Full-length scalars after/before:
  - run 2: GOST-3410-2012-256-test 0.86–1.09×; SM2 0.96–1.04×
  - run 1: GOST-3410-2012-256-test 1.29–1.55×; SM2 1.29–1.42×.

  secp256k1 and P-256 take projective ladders and are unaffected.